Seminario de Automatización

Clase 6

Recuperación aumentada e intervención humana.

Tecnicatura Universitaria en Programación
Gonzalo Bechara Balaldi

De dónde venimos

La clase pasada

- El modelo como un nodo más
- Salida estructurada
- Guardia determinista antes de preguntar
- Validar lo que devuelve

Hoy

- Le damos información que solo tenemos nosotros
- Y pasamos de clasificar a redactar

> Clasificar mal manda un mensaje a la cola equivocada. Redactar mal **le miente a un cliente en nuestro nombre**. Sube el costo del error, y por eso sube el control.

Recuperar

El modelo no sabe lo tuyo

Sabe mucho del mundo y nada de tu empresa. Tus plazos, tus precios y tus políticas son desconocidas.

¿Cómo lo podemos solucionar?

Reentrenarlo

Caro, lento, y hay que rehacerlo cada vez que cambia una política.

Meterle todo en el prompt

Funciona con muy poca información, llena a todos los prompts de contexto que quizás no use.

Recuperar lo que hace falta

Buscar los pocos fragmentos que tienen que ver con la pregunta y pasarle solo esos.

La tercera es la recuperación aumentada.

Qué es un embedding

Un modelo convierte un texto en una lista de números. Esa lista es una dirección en un espacio de muchas dimensiones.

> Dos textos que quieren decir lo mismo **apuntan para el mismo lado**, aunque no compartan ni una palabra.

"¿Cuánto tardan en llegar?" y "Plazos de entrega" no comparten nada, y sin embargo quedan cerca.

```
{
  "embedding": {
    "values": [
      0.0134, -0.0271, 0.0455,
      -0.0098, 0.0312, ...
    ]
  }
}
```

Son 768 números por fragmento. No hay nada que interpretar ahí adentro: lo único que se hace con ellos es medir ángulos.

Buscar por significado



La medida

El coseno del ángulo entre los dos vectores. Va de -1 a 1: cuanto más cerca de 1, más parecido el significado.

El mismo modelo de los dos lados

Si indexás con uno y consultás con otro, los vectores no se comparan. No da error: da resultados sin sentido.

El tamaño del fragmento

Muy chico

Una oración suelta. Recupera bien pero le falta el contexto para responder.

Justo

Una política completa y corta. Entra entera en la respuesta y se entiende sola.

Muy grande

Un documento entero. El vector queda promediado entre veinte temas y no se parece a ninguna pregunta.

Un fragmento es lo que le pasarías a una persona para que responda esa pregunta y nada más.

El umbral de recuperación

Si lo más parecido que hay en la base no se parece lo suficiente, la respuesta correcta es que no tenemos con qué responder.

Sin umbral

La búsqueda siempre devuelve los tres más cercanos, aunque los tres no tengan nada que ver. El modelo recibe basura y contesta igual.

Con umbral

Si no encontramos nada debajo del umbral contestamos que no sabemos o enviamos el caso va directo a una persona.

El número se **calibra mirando los puntajes reales**, no se elige de antemano. Depende del modelo de embeddings y del idioma.

Confiar

Alucinaciones

Es la propiedad más peligrosa de un modelo de lenguaje, y no se arregla con un prompt mejor.

Inventa la política

Le preguntan por algo que no está en la base y redacta un plazo verosímil.

Completa lo que falta

El fragmento dice la mitad y el modelo pone la otra mitad de su cosecha.

Cita una fuente que no recibió

Devuelve un identificador de fragmento que nunca estuvo en el prompt.

Elige entre dos versiones

Le pasás dos fragmentos que se contradicen y se queda con uno, sin avisar.

Ninguna es fácil de detectar. Requiere de revisión experta.

Pedirle la confianza

Entre los campos de la salida estructurada va uno más: cuán seguro está el propio modelo.

```
{
  "respuesta": "En Rosario entregamos
    dentro de las 48 horas hábiles.",
  "confianza": 0.92,
  "fragmentos_usados": ["kb-01"],
  "requiere_accion": false,
  "falta_informacion": false
}
```

No es una medida exacta, y aun así separa bastante bien lo que se puede mandar de lo que hay que mirar.

Un modelo puede estar muy seguro y equivocado. La confianza es **una señal más**, no una garantía.

El control que atrapa la invención

De las cuatro formas de inventar, hay una que se detecta con una línea de código.

Lo que se le pasó

El flujo sabe exactamente qué fragmentos le puso en el prompt: `kb-01`, `kb-06`, `kb-05`.

Lo que dice haber usado

Viene en `fragmentos_usados`. Si aparece alguno que no está en la lista de arriba, la respuesta no está respaldada.

Que cite bien no prueba que la respuesta sea correcta. Que cite mal **sí prueba** que está alucinando.

Cuando la base se contradice

Las políticas cambian. Las versiones viejas quedan en la carpeta compartida.

Versión anterior

15 días para devolver. El envío lo paga el cliente.

Versión vigente

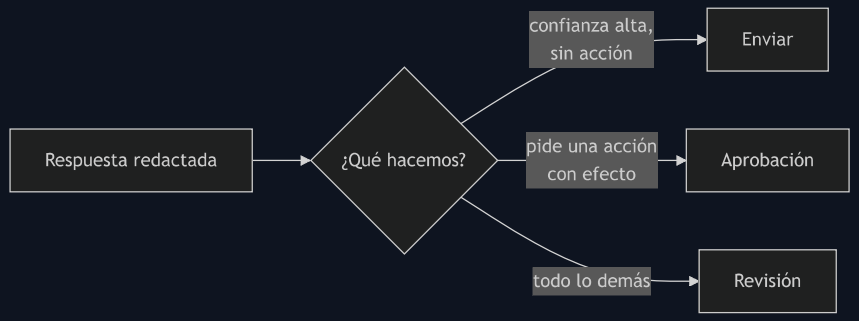
30 días para devolver. El envío lo paga la empresa.

Las dos están en la base y las dos se recuperan juntas. El modelo **no tiene que elegir**: tiene que avisar que hay una contradicción y bajar la confianza.

La fecha de vigencia va en cada fragmento por esto. No para que el modelo decida, sino para que la persona que revisa vea cuál es cuál.

Intervenir

Tres destinos, no dos



Enviar

Sale solo. Es la única rama sin nadie mirando, y por eso es la más exigente.

Aprobación

El borrador está listo. Decide una persona.

Revisión

Baja confianza, contradicción, invención o salida inválida.

Acciones con efecto

Responder una consulta no cambia nada. Cancelar un pedido, devolver dinero o dar de baja, sí.

La regla

Ninguna acción con efecto se ejecuta sin que una persona la apruebe, por más alta que sea la confianza.

Y mientras tanto

El borrador se redacta igual. La persona aprueba o corrige en vez de escribir desde cero.

La automatización no desaparece cuando hace falta una persona: **le deja el trabajo hecho** y se queda esperando la aprobación.

La cola de revisión es parte del producto

Una cola que no sirve

- Llega el correo crudo, sin nada más
- No dice por qué cayó ahí
- No trae el borrador
- Nadie la mira a los tres días

Una cola que sirve

- El motivo, escrito
- Los fragmentos que se recuperaron, con su puntaje
- El borrador que el modelo propuso
- Un botón de aprobar y uno de corregir

> Si la cola no se mira, el sistema no tiene revisión humana: Es un **tacho de basura con otro nombre**.

Caso de uso

Un asistente de correos

La misma tienda de la clase pasada. Ahora hay que contestarles, no solo clasificarlos.

La base de conocimiento

Doce fragmentos de políticas: entregas, devoluciones, garantía, pagos, facturación, sucursales, descuentos, bajas.

Dos de los doce se contradicen entre sí.

Los correos

Nueve consultas reales. Algunas están cubiertas por la base, otras no, y una pide una acción con efecto.

Uno intenta darle órdenes al asistente.

Del otro lado hay un cliente leyendo. Una respuesta inventada nos puede generar problemas legales. ¿Qué pasa si le decimos al cliente que el envío es gratis y luego no lo es?

Qué hay que producir

1	Un `índice`: cada fragmento con su vector	una vez
2	Para cada correo, los `tres fragmentos` más cercanos y su puntaje	recuperación
3	Un `umbral` calibrado con los puntajes que ven, no elegido de antemano	compuerta
4	Una `respuesta redactada` solo con lo que dicen esos fragmentos	generación
5	Tres colas: `enviar`, `aprobación` y `revisión`, con el motivo escrito	ruteo

> La pregunta con la que se cierra el ejercicio: de los nueve, **¿a cuántos le mandarías la respuesta sin leerla?**

Práctica guiada

Paso 0 — Dos credenciales, una clave

a	La misma clave gratuita de AI Studio de la clase pasada	ya la tienen
b	Credencial `Google Gemini (PaLM) API` para el subnodo del modelo	ya la tienen
c	Credencial `Header Auth` nueva: nombre `x-goog-api-key`, valor la misma clave	nueva
d	Seleccionarla en los dos nodos de embeddings	listo

> Son dos credenciales porque el nodo HTTP no usa el mismo tipo que el subnodo del modelo. **La clave es una sola.** (Es un tecnicismo de N8N)

Paso 1 — Indexar

a	Ejecutar el template y abrir la salida de `Embeber fragmentos`	mirar
b	Contar: doce items, y 768 números en cada uno	verificar
c	Armar el índice: cada fragmento con su vector, en un solo item	construir

> El nodo HTTP devuelve los vectores **en el mismo orden** en que recibió los fragmentos, y nada más: el texto no viaja de vuelta. Por eso se emparejan por posición.

> Si las dos listas no miden lo mismo, algo se perdió. Conviene que el nodo se queje en vez de seguir con el índice mal armado.

Paso 2 — Recuperar y calibrar

Primero, sin umbral

Calcular la similitud de cada correo contra los doce fragmentos y mirar **todos** los puntajes, no solo los tres mejores.

Después, el corte

Con esos números a la vista, elegir dónde cortar. Lo relevante y lo que no se separan bastante bien.

Hay un correo que dice solo "hola" y otro que pregunta por un medio de pago que no está en la base. **Esos dos tienen que quedar por debajo del corte**, y son la prueba de que el umbral está bien puesto.

Paso 3 — Responder con lo nuestro

Los fragmentos recuperados se pegan dentro del prompt. Eso es toda la recuperación aumentada.

En el prompt del sistema

Responder **solo** con lo que digan los fragmentos. Si no alcanzan, decirlo. Si se contradicen, avisar. No inventar.

En el mensaje

El correo del cliente y abajo los tres fragmentos, cada uno con su identificador y su fecha de vigencia.

Los identificadores van en el prompt por una razón concreta: sin ellos, el modelo no puede decir cuáles usó, y **perdemos el único control mecánico** que tenemos.

Paso 4 — Validar y rutear

a	¿La confianza está entre 0 y 1? ¿La respuesta no está vacía?	forma
b	¿Los fragmentos que dice haber usado `estaban` entre los que le pasamos?	fuente
c	¿Respondió sin citar nada y sin admitir que le faltaba información?	respaldo
d	Rutear: aprobación primero, enviar después, revisión para todo lo demás	orden

> Al terminar, revisar la cola de **enviar** correo por correo. Esa es la única rama que le habla a un cliente sin que nadie mire.

Lo que probablemente falle

Todo cae en revisión
El umbral quedó muy alto. Mirar los puntajes reales antes de elegirlo.

Todo se envía
El umbral quedó muy bajo, o el ruteo mira la confianza y nada más.

Las respuestas no tienen que ver
El índice quedó desfasado: el fragmento 3 con el vector del 4.

Todos los puntajes son parecidos
Se indexó con un modelo y se consultó con otro, o se embebió el objeto entero en vez del texto.

El de "hola" recibió respuesta
Un texto corto igual produce un vector. El umbral es lo único que lo frena.

El DNI del cliente apareció en la respuesta
Falta la línea del prompt que prohíbe repetir datos personales.

Cierre

Hoy vimos

- Buscar primero y responder después
- Embeddings y búsqueda por significado, escritos a mano
- Tres compuertas: umbral, confianza y fuentes citadas
- Que las acciones con efecto siempre pasan por una persona
- Que la cola de revisión es parte del producto

Qué falló durante la práctica

> Repaso en vivo de los errores que aparecieron hoy en el aula.

> **La pregunta del principio**
>
> De los nueve correos, ¿a cuántos les mandarías la respuesta sin leerla? La respuesta de cada uno dice más sobre el sistema que cualquier métrica.